

Cálculo del vector de personalización y del ranking PageRank

Especificación paso a paso del algoritmo de relevancia adaptativa del gestor documental, con un caso trabajado completo que permite reproducir cada valor a mano, sin ejecutar el código.

Ámbito: PageRank Personalizado (PPR) Ejemplo: escenario B1 (9 nodos) Factor de amortiguación: $\alpha = 0.85$

§1 Objeto y alcance

Este documento describe, de forma autocontenida y reproducible, cómo el sistema construye el **vector de teletransporte personalizado** $\mathbf{v}$ y cómo, a partir de él, obtiene el **ranking de relevancia** $\mathbf{r}_v$ de cada nodo mediante PageRank Personalizado.

El objetivo es que cualquier persona pueda reproducir los resultados con una calculadora, verificando cada cifra intermedia. Se utiliza como hilo conductor el escenario de prueba **B1**: un usuario valora dos categorías (*Terror gótico* con 2★ y *Crimen* con 5★) y no valora ningún libro individual.

§2 El grafo sobre el que opera el algoritmo

Nodos, aristas y pesos de la estructura de cálculo.

El catálogo se modela como un grafo dirigido. En el escenario B1, tras podar las ramas vacías y colapsar las cadenas de un solo hijo, quedan **9 nodos**: la raíz *Mystery&Crime fiction*, las dos categorías *Terror gótico* y *Crimen*, y 6 libros hoja (3 por categoría).

Antes de ejecutar PageRank, cada arista jerárquica se duplica en sentido inverso, de modo que el grafo de cálculo es **bidireccional** (padre↔hijo). Cada arista lleva un **peso**:

Peso 1 — aristas entre nodos intermedios (raíz ↔ categoría).

Peso 3 — aristas que tocan un libro (categoría ↔ libro). Es el parámetro `peso_libros`.

En B1 no hay aristas de referencia (libros similares), porque los 6 libros tienen `tiene_similar = false`.

EN EL CÓDIGO

El grafo bidireccional con pesos se construye en `logica.py` → `version_personalizacion_likes()`, al recorrer las aristas y añadir `(u,v)` y `(v,u)`. La poda que deja los 9 nodos la realiza `lector.py` → `_simplificar_arbol()`. Los nombres «Crimen» y «Terror gótico» son alias de presentación definidos en `server.py`; los identificadores internos son `.../Crime` y `.../Suspense`.

§3 Notación y constantes globales

El punto de partida de cada nodo es un **score base**. Para los libros, ese score es el **promedio ponderado bayesiano**, que evita que un libro con muy pocos votos y media perfecta distorsione el ranking:

PROMEDIO PONDERADO BAYESIANO DE UN LIBRO

$$\text{Score} = (N / (N + m)) \cdot R + (m / (N + m)) \cdot C$$

R — valoración media del libro (`average_rating`), salvo que el usuario haya valorado ese libro concreto, en cuyo caso **R** pasa a ser su puntuación.

N — número de valoraciones del libro (`ratings_count`). *Nota:* en la fórmula clásica esta variable se llama **v**; aquí la nombramos **N** para no confundirla con el vector de teletransporte **v**.

C — **media aritmética simple** de los `average_rating` de todos los libros del catálogo de referencia (no ponderada por votos).

m — umbral de votos: se ordenan los `ratings_count` de menor a mayor y se toma el de la posición `int(nº_libros × 0.5)` (percentil 50).

Para el escenario B1, con sus 6 libros:

RESULTADO — CONSTANTES B1

$$C = (4.22 + 3.90 + 3.91 + 4.18 + 3.97 + 4.15) / 6 = 24.33 / 6 = 4.055$$

Votos ordenados: [11716, 18405, 20080, 24621, 26929, 67230]. Posición `int(6 × 0.5) = 3` → $m = 24621$ (los votos de *Blue Smoke*).

EN EL CÓDIGO

`_bayesian_scores()` calcula C , m y el score de cada libro. La sustitución de R por la valoración del usuario ocurre antes, en `ratings_efectivos()`.

§4 Construcción del vector de teletransporte (antes de normalizar)

Cómo se obtiene el valor de cada nodo según su tipo y según si el usuario lo ha valorado.

El vector sin normalizar asigna a cada nodo un valor $w = \text{score base} \times \text{peso_libros} \times 10$ (es decir, $\times 30$). Como **todos** los nodos se multiplican por el mismo factor 30, éste se cancela al normalizar (§5): no altera el vector final, pero se incluye aquí para que las cifras intermedias coincidan exactamente con las del sistema.

El **score base** se determina así, según el tipo de nodo:

a) Libro que el usuario no ha valorado

Su score base es su promedio bayesiano (§3), con $R = \text{average_rating}$. Es el caso de los 6 libros de B1.

b) Libro que el usuario sí ha valorado

Se aplica **la misma fórmula bayesiana**, pero con $R =$ número de estrellas del usuario en lugar del `average_rating`. El código sustituye R pero **conserva el mismo N** del libro, de modo que la valoración **se suaviza siempre igual que la de cualquier otro libro**: queda ponderada por $N/(N+m)$ y arrastrada hacia C por $m/(N+m)$, sin saltar directamente a las estrellas puestas. (*En el escenario B1 no llega a activarse, porque el usuario solo valora categorías y no libros; pero el sistema se comporta así siempre que existe una valoración de libro.*)

EN EL CÓDIGO — DÓNDE SE SUAVIZA

`ratings_efectivos()` fija R = valoración del usuario y mantiene N ; después `_bayesian_scores()` aplica $(N/(N+m)) \cdot R + (m/(N+m)) \cdot C$ a ese libro exactamente igual que a los demás. Dos modos son la excepción y **no** suavizan: con `aplicar_prior = False` (modo sin corrección bayesiana) se usa la estrella en crudo, y en `debug_mode` el vector toma directamente el número de estrellas del usuario.

c) Categoría (o raíz) que el usuario no ha valorado

Su score base es la **media de los scores base de sus hijos directos**, calculada de abajo hacia arriba (primero los libros, luego las categorías, luego la raíz).

d) Categoría que el usuario sí ha valorado

Su score base es directamente el **número de estrellas en crudo** (1–5). Este valor **sustituye** a la media por defecto; no se suma a ella ni se suaviza. En B1: *Terror gótico* → 2 y *Crimen* → 5.

EN EL CÓDIGO

Los scores base de categorías y raíz los calcula `_score_por_defecto()` (recorrido en postorden: hijos antes que padres). El vector se ensambla en `version_personalizacion_likes()`: primero `base × peso_libros × 10` para todos los nodos; después, la línea que recorre `user_genre_ratings` **sustituye** el valor de las categorías valoradas por `estrellas × peso_libros × 10`.

§5 Normalización

El vector se normaliza **dividiendo cada componente por la suma de todas las componentes** (normalización L1). No se aplica softmax ni ninguna transformación exponencial.

VECTOR DE TELETRANSPORTE NORMALIZADO

$$v_i = w_i / \sum_j w_j$$

Puesto que el factor común $\times 30$ aparece en todos los w_i , se cancela en el cociente: normalizar los valores en crudo `[4.1757, ..., 2, 5, 4.0621]` produce exactamente el mismo vector que normalizar los valores $\times 30$.

§6 Ejemplo trabajado: escenario B1

Todos los valores intermedios, desde los datos de entrada hasta el vector normalizado.

Paso 1 — Score bayesiano de cada libro

Con $C = 4.055$ y $m = 24621$, aplicando la fórmula del §3 a cada libro:

LIBRO	R	N (VOTOS)	SCORE BASE
A Scandal in Bohemia	4.22	67 230	4.175771
The Obsession	4.18	26 929	4.120299
Envy	4.15	11 716	4.085631
Blue Smoke	3.97	24 621	4.012500
Hercule Poirot's Christmas	3.91	18 405	3.992974
Cards on the Table	3.90	20 080	3.985366

Tabla 1. Score bayesiano de los 6 libros. Blue Smoke tiene $N = m$, de ahí su score = media exacta de R y C .

Ejemplo de cálculo (A Scandal in Bohemia):

$$(67230 / 91851) \cdot 4.22 + (24621 / 91851) \cdot 4.055 = 0.731946 \cdot 4.22 + 0.268054 \cdot 4.055 = 4.175771$$

Paso 2 — Score base de categorías y raíz

Media de los scores base de los hijos directos (§4c):

RESULTADO — SCORES POR DEFECTO

Terror gótico = media(4.120299, 4.012500, 4.085631) = **4.072810**

Crimen = media(4.175771, 3.992974, 3.985366) = **4.051370**

Mystery&Crime fiction (raíz) = media(4.072810, 4.051370) = **4.062090**

La raíz se calcula con los scores **por defecto** de las categorías (4.0728 y 4.0514), no con los valores valorados por el usuario (2 y 5). Por eso la raíz vale 4.0621 y no la

media de 2 y 5.

Paso 3 — Vector sin normalizar (× 30) y aplicación de las valoraciones

Cada score base se multiplica por 30. Las dos categorías valoradas sustituyen su valor por `estrellas × 30` : *Terror gótico* (2★) → 60, *Crimen* (5★) → 150.

NODO	ORIGEN DEL VALOR	W = BASE × 30
Crimen	5★ del usuario → 5 × 30	150.00000
A Scandal in Bohemia	bayesiano × 30	125.27313
The Obsession	bayesiano × 30	123.60897
Envy	bayesiano × 30	122.56893
Mystery&Crime fiction	media hijos × 30	121.86270
Blue Smoke	bayesiano × 30	120.37500
Hercule Poirot's Christmas	bayesiano × 30	119.78922
Cards on the Table	bayesiano × 30	119.56098
Terror gótico	2★ del usuario → 2 × 30	60.00000
Σ total		1063.03893

Tabla 2. Vector de teletransporte sin normalizar. Las filas de categoría con valoración del usuario están resaltadas.

Paso 4 — Normalización (÷ 1063.039)

Cada componente dividida por la suma total da el vector de teletransporte definitivo:

NODO	W	V = W / Σ
Crimen	150.00000	0.141105
A Scandal in Bohemia	125.27313	0.117844
The Obsession	123.60897	0.116280
Envy	122.56893	0.115300
Mystery&Crime fiction	121.86270	0.114636
Blue Smoke	120.37500	0.113237
Hercule Poirot's Christmas	119.78922	0.112686
Cards on the Table	119.56098	0.112471
Terror gótico	60.00000	0.056442

Tabla 3. Vector de teletransporte normalizado $\mathbf{v}$ (suma = 1).

§7 Del vector de teletransporte al ranking final

Con el vector $\mathbf{v}$ ya determinado, el ranking se obtiene aplicando iterativamente la ecuación de PageRank Personalizado hasta la convergencia:

PAGERANK PERSONALIZADO

$$\mathbf{r_v} = \alpha \cdot \mathbf{P}^T \mathbf{r_v} + (1 - \alpha) \cdot \mathbf{v}$$

$\mathbf{P}$ es la matriz de transición del grafo bidireccional: cada fila (nodo origen) reparte probabilidad 1 entre sus vecinos, proporcionalmente al peso de cada arista (1 entre intermedios, 3 hacia libros). Cada libro tiene una única arista de salida hacia su categoría, por lo que su fila vale 1 en esa columna. Con $\alpha = 0.85$, el algoritmo converge a:

PUESTO	NODO	R_v
1	Crimen	0.255563
2	Terror gótico	0.219915
3	A Scandal in Bohemia	0.082845
4	Hercule Poirot's Christmas	0.082071
5	Cards on the Table	0.082039
6	The Obsession	0.073520
7	Envy	0.073373
8	Blue Smoke	0.073063
9	Mystery&Crime fiction	0.057611

Tabla 4. Ranking de convergencia r_v para el escenario B1 ($\alpha = 0.85$).

El resultado es coherente con la valoración del usuario: *Crimen* (5★) y todos sus libros ascienden, mientras que *Terror gótico* (2★) y los suyos descienden, aunque la categoría conserva un puesto alto por su posición estructural en el grafo.

EN EL CÓDIGO

La iteración la resuelve `nx.pagerank(G, alpha=0.85, personalization=v, weight='weight')`. NetworkX construye P internamente a partir del grafo con pesos y usa v tanto como vector de teletransporte como para los nodos sin salida.

§8 Receta de reproducción manual

Procedimiento completo para reproducir cualquier escenario con una calculadora:

1

Calcular las constantes globales

C = media simple de los `average_rating`. m = valor en el percentil 50 de los `ratings_count` ordenados.

2

Score base de cada libro

Aplicar la fórmula bayesiana. Si el usuario valoró el libro, usar sus estrellas como R ; en caso contrario, su `average_rating`.

3

Score base de categorías y raíz

De abajo arriba: cada categoría = media de sus hijos; la raíz = media de las categorías. Si el usuario valoró una categoría, sustituir ese valor por su número de estrellas.

4

Vector sin normalizar

Multiplicar cada score base por `peso_libros × 10 = 30` (opcional: el factor se cancela al normalizar).

5

Normalizar

Dividir cada componente por la suma total. Se obtiene el vector de teletransporte $\mathbf{v}$.

6

Ejecutar PageRank

Iterar $\mathbf{r}_v = \alpha \cdot \mathbf{P}^T \mathbf{r}_v + (1 - \alpha) \cdot \mathbf{v}$ con $\alpha = 0.85$ hasta la convergencia.

§9 Correspondencia con el código fuente

PASO	FUNCIÓN / FICHERO
Sustitución de R por la valoración del usuario (libros)	<code>ratings_efectivos()</code> · logica.py
Constantes C, m y score bayesiano de libros	<code>_bayesian_scores()</code> · logica.py
Score base de categorías y raíz (media de hijos)	<code>_score_por_defecto()</code> · logica.py
Ensamblado y normalización del vector · × 30 · sustitución por estrellas	<code>version_personalizacion_likes()</code> · logica.py
Construcción del grafo bidireccional con pesos	<code>version_personalizacion_likes()</code> · logica.py
Poda del árbol hasta los 9 nodos del ejemplo	<code>_simplificar_arbol()</code> · lector.py
Ejecución iterativa de PageRank	<code>nx.pagerank()</code> (NetworkX)

Tabla 5. Ubicación de cada paso en el código.